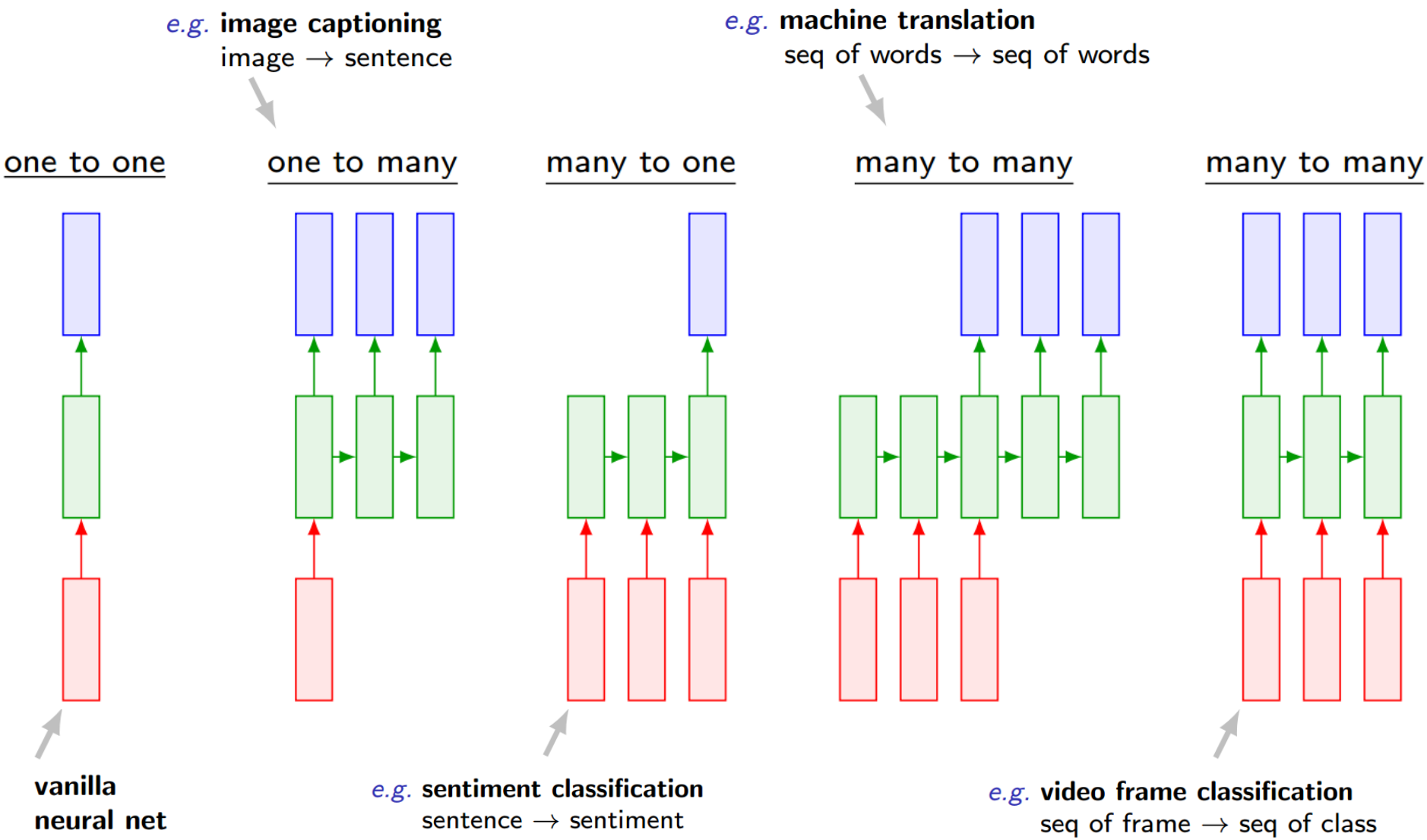
CSE4007
Artificial Intelligence

Lecture 14: Large Language Models (LLMs)

Department of Data Science
Hanyang University

Recap: RNN modeling capability

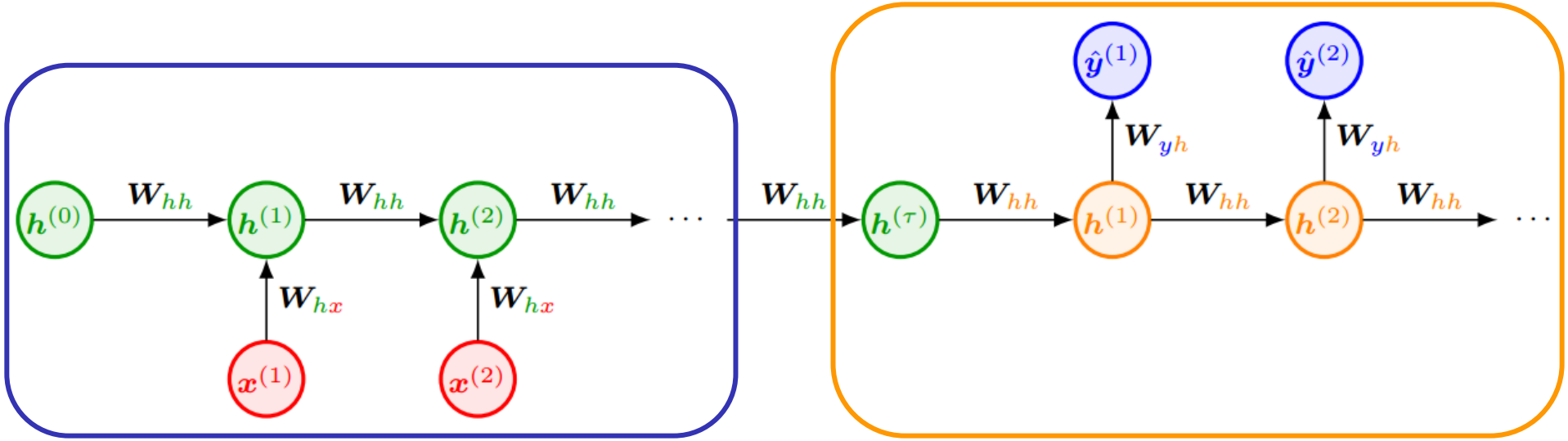


Recap: Sequence-to-sequence

- = many-to-one + one-to-many

- many-to-one
  - Encode input sequence into a single vector

- one-to-many
  - Produce output sequence from the single vector



Encoder Decoder

Recall: Sequence-to-sequence with Transformer

- encoding component: a stack of encoders
- decoding component: a stack of decoders of the same number

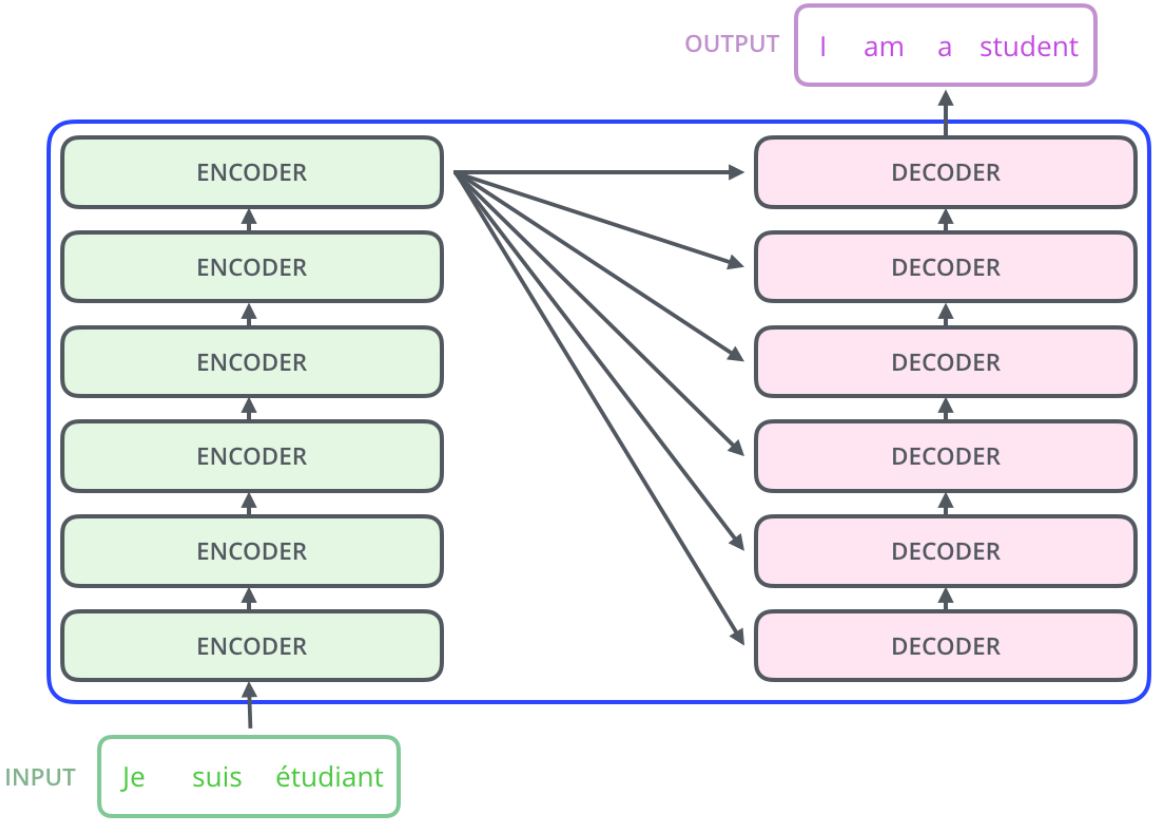


image source: Jay Alammar, The Illustrated Transformer, http://jalammar.github.io/illustrated-transformer

Recall: Transformer Architecture

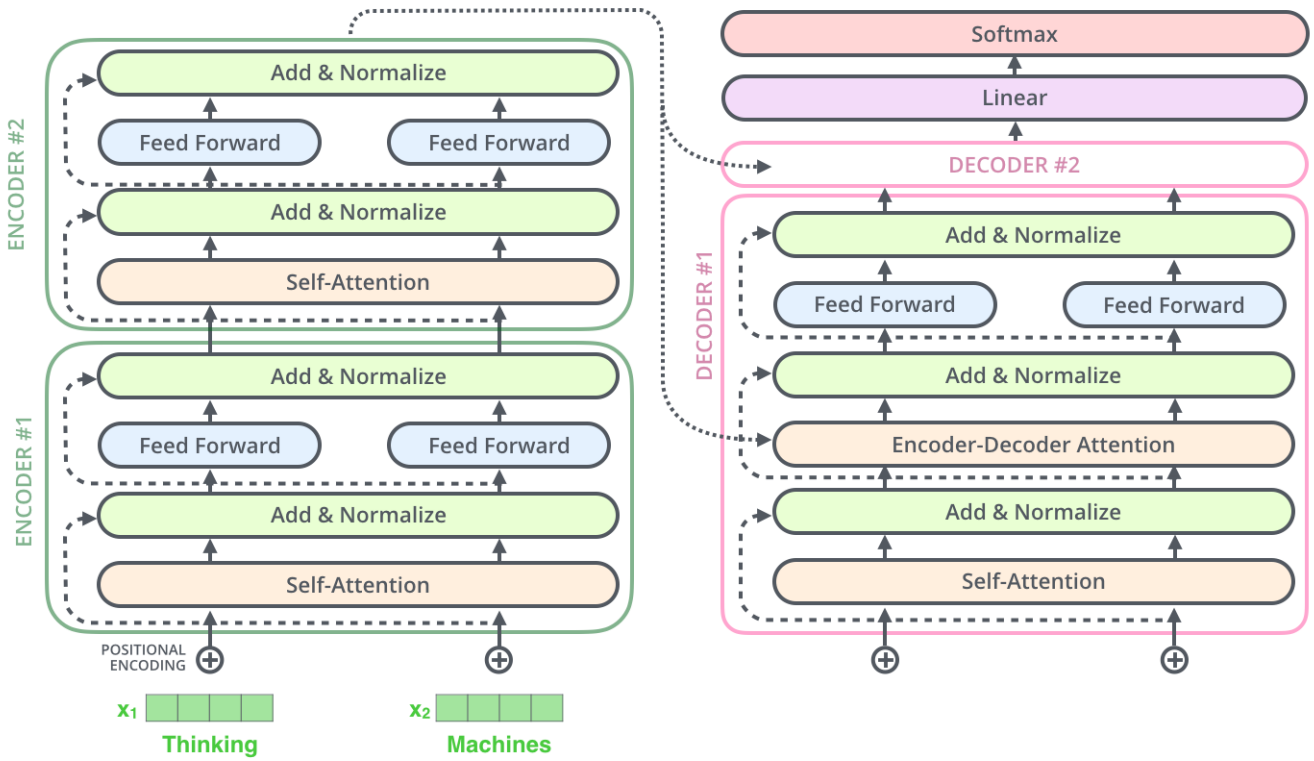
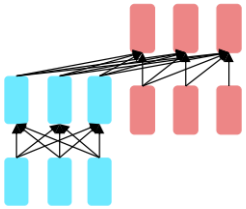
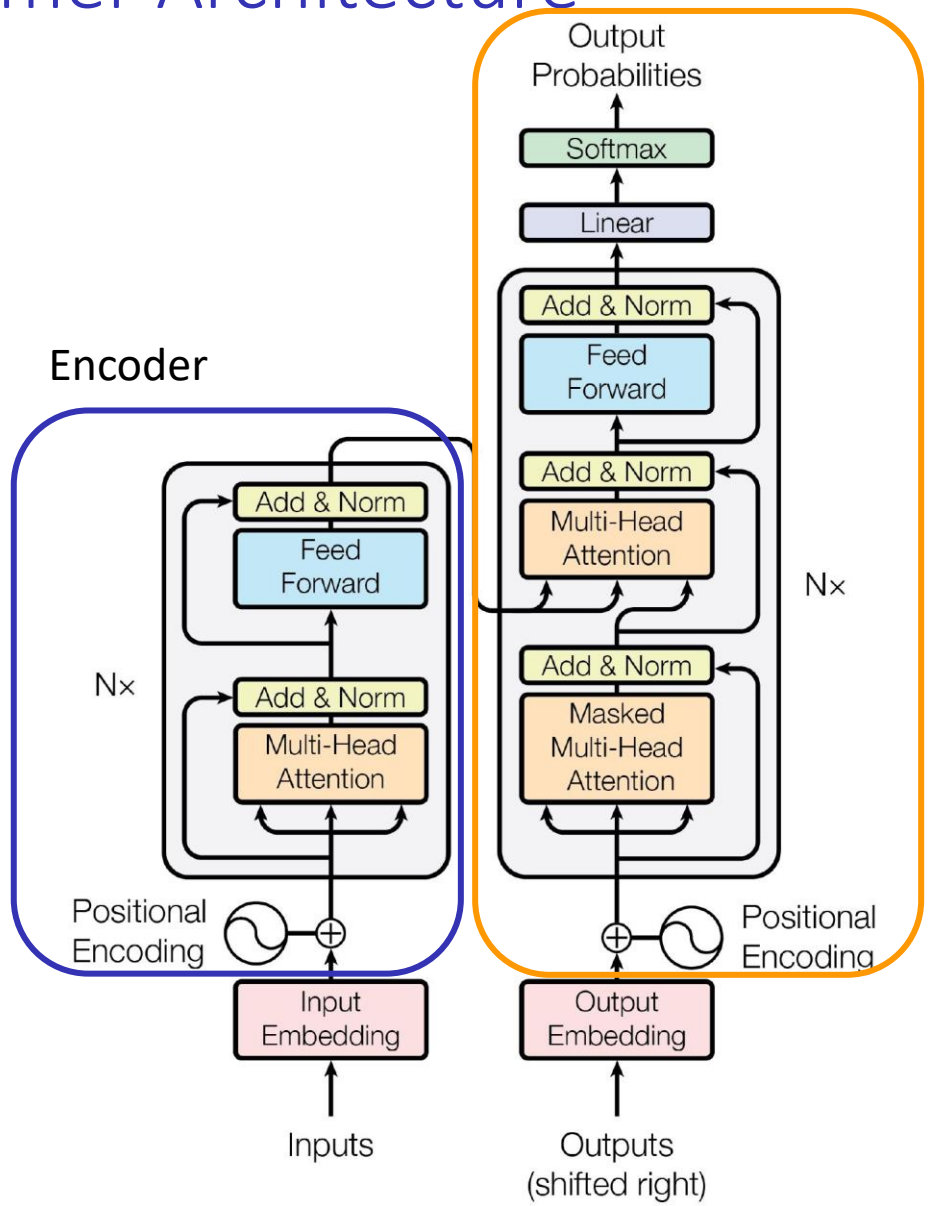


image source: Jay Alammar, The Illustrated Transformer, http://jalammar.github.io/illustrated-transformer

Recall: Transformer Architecture

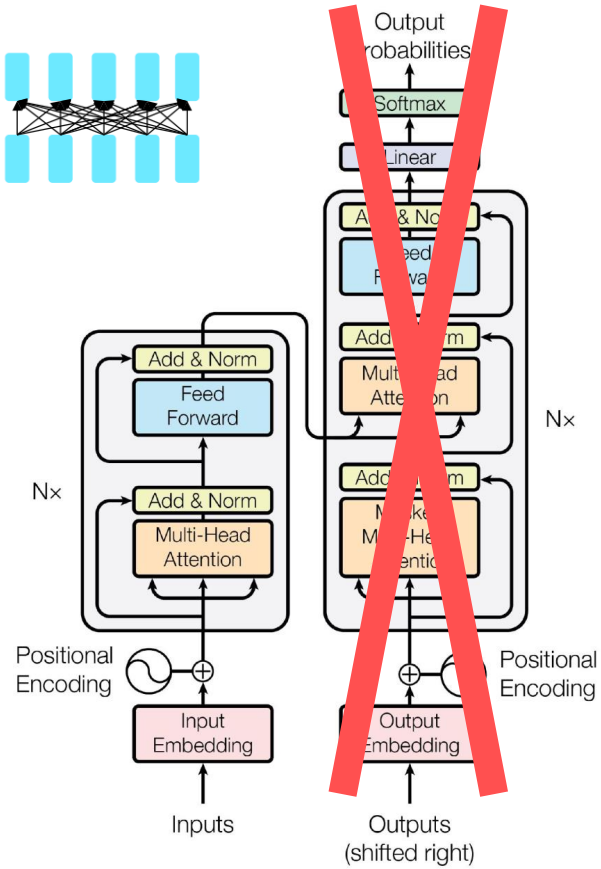
e.g., Flan-T5, Whisper





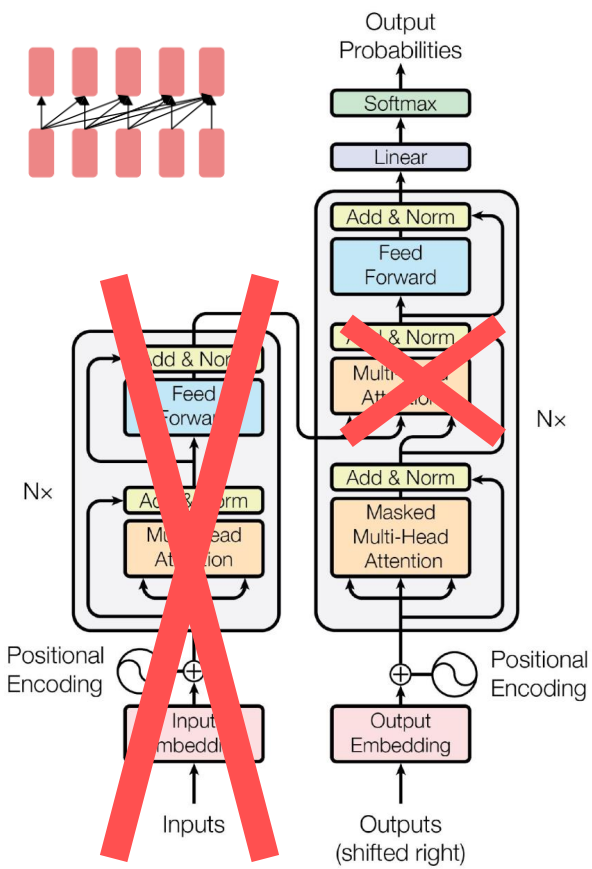
Recall: BERT vs GPT

BERT: Encoder part of Transformer

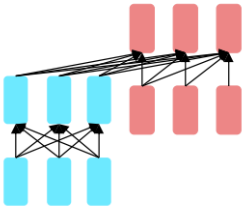


GPT: Decoder part of Transformer

Claude, Llama, Mixtral ,etc.



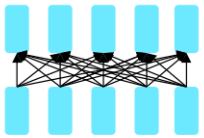
Encoder-Decoder



- Trained to map from one sequence to another

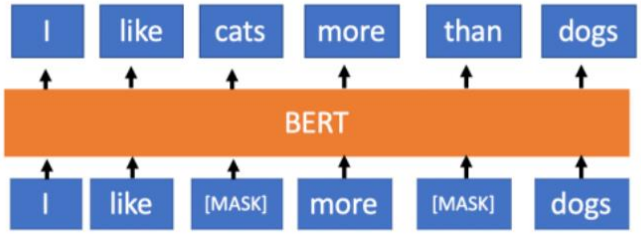
- Very popular for:
  - Machine translation (map from one language to another)
  - Speech recognition (map from acoustics to words)

BERT family

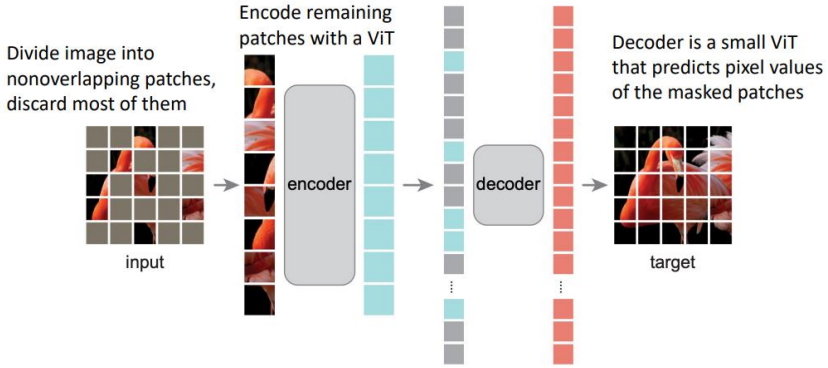


- Masked Language Models (MLMs)

- Trained by predicting words from surrounding words on both sides

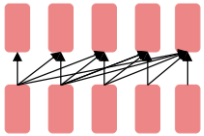


- Recall: Masked Autoencoder (MAE)
  - Vision counterpart of MLM



Decoder-only Models

- Idea: many NLP tasks can be turned into tasks of predicting next words
  - Often called next token prediction
  - Predict words left to right

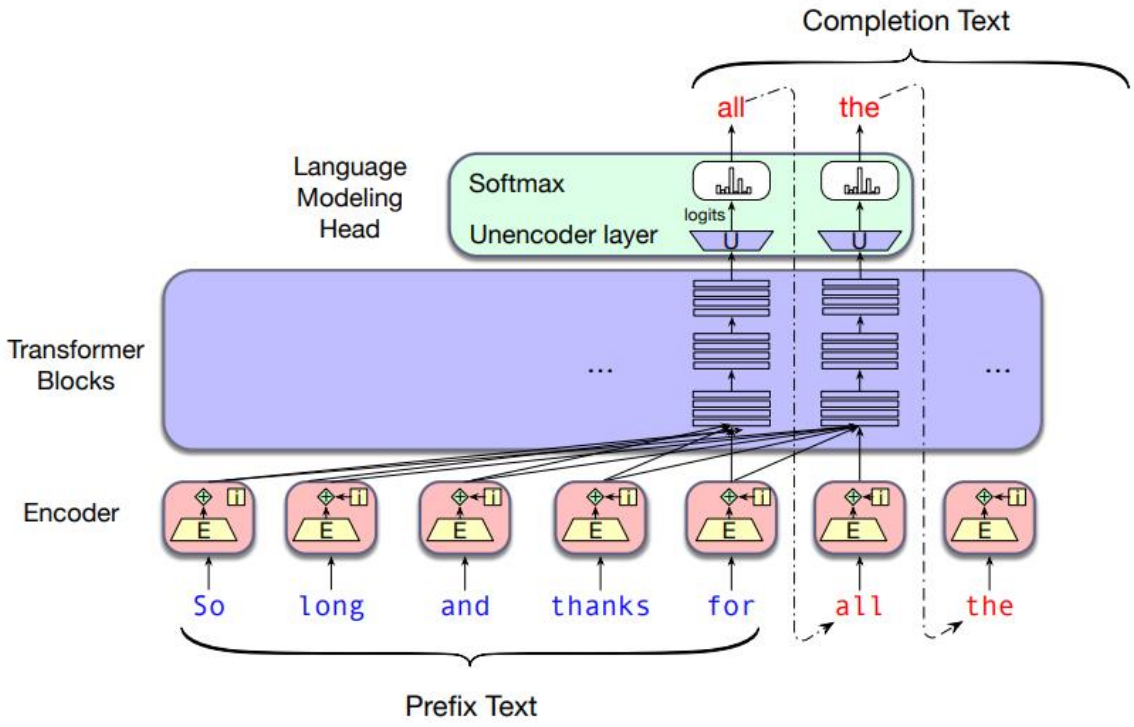


- Decoder-only models are also called:
  - Causal LLMs
  - Autoregressive LLMs
  - Left-to-right LLMs

Conditional Generation

- Generating text conditioned on previous text

- Can be considered as special form of masked language modeling



Many practical NLP tasks can be cast as word prediction!

- Sentiment analysis task: e.g., “I like Jackie Chan”

- Step 1. We give the language model this string:
  `The sentiment of the sentence "I like Jackie Chan" is:`

- Step 2. And see what word it thinks comes next:

$$P(\text{positive}|\texttt{The sentiment of the sentence ``I like Jackie Chan" is:})$$
$$P(\text{negative}|\texttt{The sentiment of the sentence ``I like Jackie Chan" is:})$$

Framing lots of tasks as conditional generation

- QA task: e.g., “I like Jackie Chan”

- Step 1. We give the language model this string:

```
Q: Who wrote the book ‘‘The Origin of Species"?  A:
```

- Step 2. And see what word it thinks comes next:

$P(w|$`Q: Who wrote the book ‘‘The Origin of Species"?  A:`$)$

- Step 3: And iterate

$P(w|$`Q: Who wrote the book ‘‘The Origin of Species"?  A: Charles`$)$

Summarization

Original

The only thing crazier than a guy in snowbound Massachusetts boxing up the powdery white stuff and offering it for sale online? People are actually buying it. For $89, self-styled entrepreneur Kyle Waring will ship you 6 pounds of Boston-area snow in an insulated Styrofoam box – enough for 10 to 15 snowballs, he says.

But not if you live in New England or surrounding states. "We will not ship snow to any states in the northeast!" says Waring's website, ShipSnowYo.com. "We're in the business of expunging snow!"

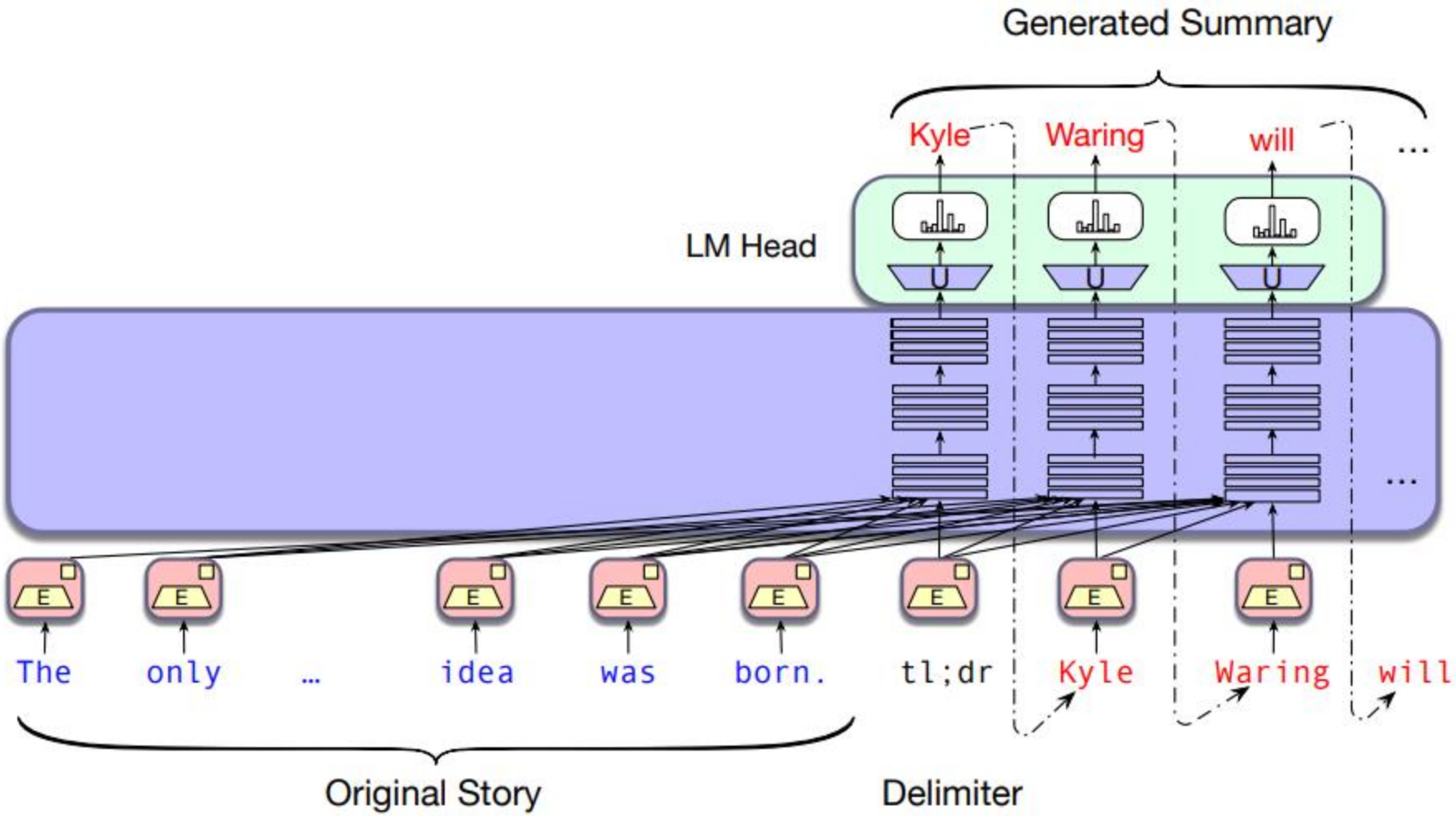
His website and social media accounts claim to have filled more than 133 orders for snow – more than 30 on Tuesday alone, his busiest day yet. With more than 45 total inches, Boston has set a record this winter for the snowiest month in its history. Most residents see the huge piles of snow choking their yards and sidewalks as a nuisance, but Waring saw an opportunity.

According to Boston.com, it all started a few weeks ago, when Waring and his wife were shoveling deep snow from their yard in Manchester-by-the-Sea, a coastal suburb north of Boston. He joked about shipping the stuff to friends and family in warmer states, and an idea was born. [...]

Summary

Kyle Waring will ship you 6 pounds of Boston-area snow in an insulated Styrofoam box – enough for 10 to 15 snowballs, he says. But not if you live in New England or surrounding states.

LLMs for summarization (using tl;dr)



Decoding and Sampling

- This task of choosing a word to generate based on the model's probabilities is called **decoding**.

- The most common method for decoding in LLMs: **sampling**.

- Sampling from a model's distribution over words:
  - choose random words according to their probability assigned by the model.
- After each token, we'll sample words to generate according to *their probability conditioned on our previous choices*

- A transformer language model will give the probability

Random Sampling

$$
\begin{aligned}
&\mathrm{i} \leftarrow 1 \\
&w_i \sim \mathrm{p}(w) \\
&\textbf{while } w_i \text{ != EOS} \\
&\quad \mathrm{i} \leftarrow \mathrm{i} + 1 \\
&\quad w_i \sim \mathrm{p}(w_i \mid w_{<i})
\end{aligned}
$$

Random sampling doesn't work very well

- Even though random sampling mostly generate sensible, high-probable words
- There are many odd, low- probability words in the tail of the distribution
- Each one is low-probability but added up, they constitute a large portion of the distribution.
- So they get picked enough, generating weird sentences

Factors in word sampling: **quality** and **diversity**

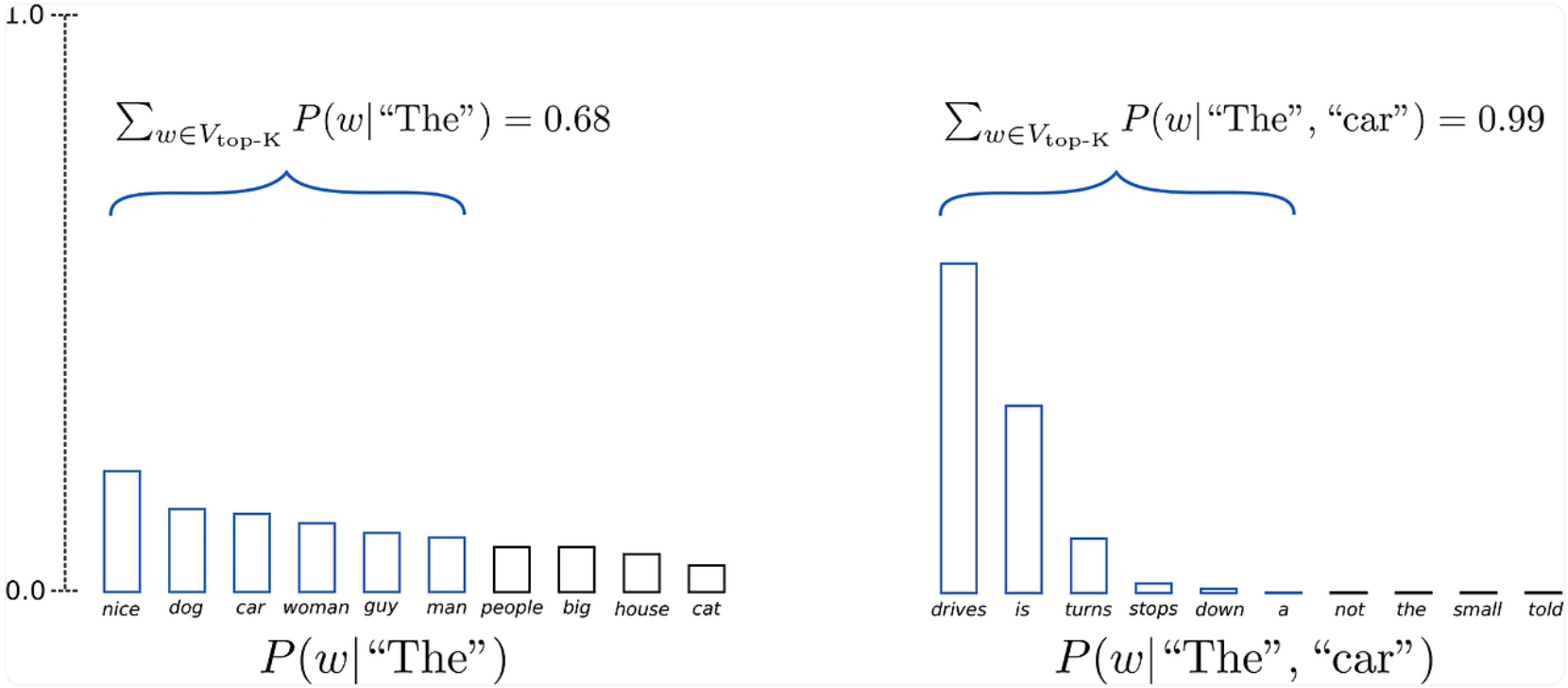
- Emphasize **high-probability** words
  - + **quality**: more accurate, coherent, and factual
  - - **diversity**: boring, repetitive

- Emphasize **middle-probability** words
  - + **diversity**: more creative, diverse
  - - **quality**: less factual, incoherent

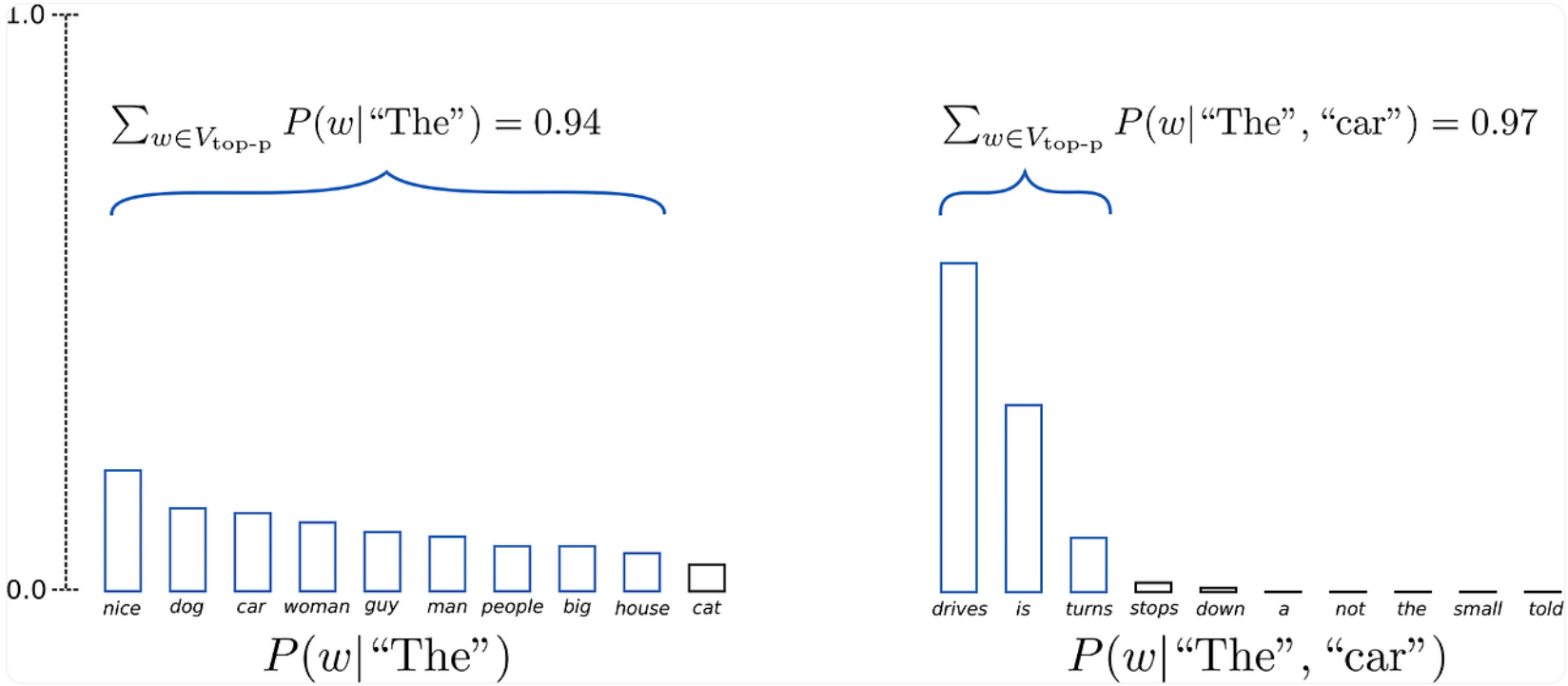
Top-k sampling

- 1. Choose # of words k
- 2. For each word in the vocabulary V, use the language model to compute the likelihood of this word given the context $p(w_t|\boldsymbol{w}_{<t})$
- 3. Sort the words by likelihood, keep only the top k most probable words.
- 4. Renormalize the scores of the k words to be a legitimate probability distribution.
- 5. Randomly sample a word from within these remaining k most probable words according to its probability.

Top-k sampling



Top-k sampling



Top-p sampling (nucleus sampling)

- Problem with top-k: k is fixed so may cover very different amounts of probability mass in different situations

- Idea: Instead, keep the top *p* percent of the probability mass

- Given a distribution $P(w_t|\boldsymbol{w}_{<t})$, the top-p% vocabulary $V(p)$ is the smallest set of words such that

$$\sum_{w \in V^{(p)}} P(w|\mathbf{w}_{<t}) \geq p$$

Temperature scaling

- Reshape the distribution instead of truncating it
- Intuition from thermodynamics:
  - a system at high temperature is flexible and can explore many possible states
  - a system at lower temperature is likely to explore a subset of lower energy (better) states.

- In low-temperature sampling ($\tau \leq 1$), we smoothly
  - increase the probability of the most probable words
  - decrease the probability of the rare words

Temperature scaling

- Divide the logit by a temperature parameter τ before passing it through the softmax

- Instead of

$$\cancel{\mathbf{y} = \mathrm{softmax}(u)}$$

- We do

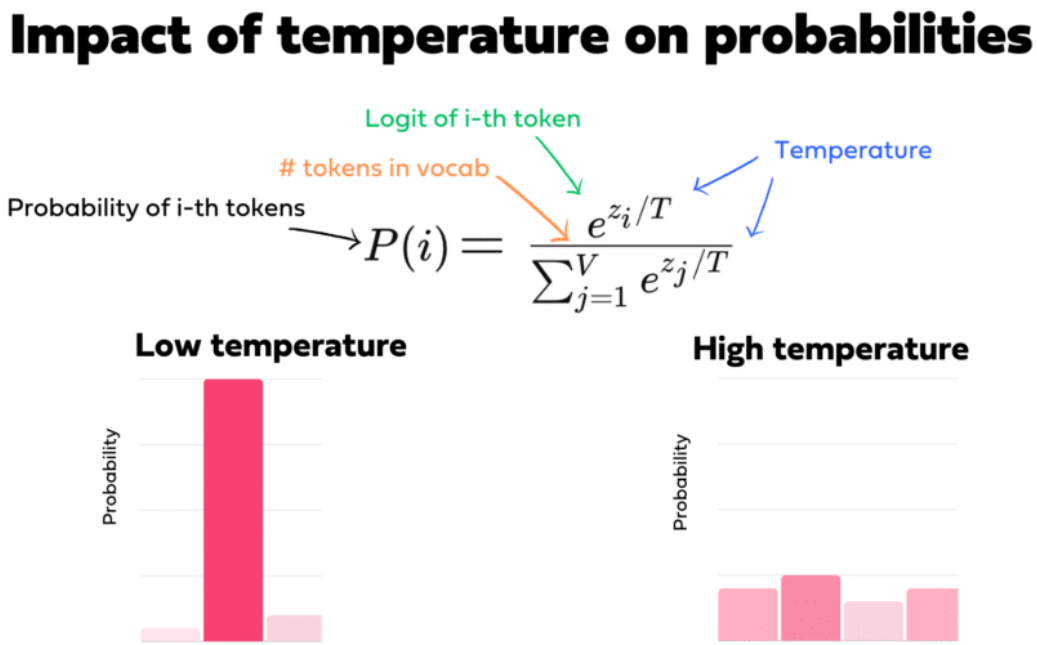
$$\mathbf{y} = \mathrm{softmax}(u/\tau)$$

Temperature scaling

$$\mathbf{y} = \text{softmax}(u/\tau) \quad 0 \leq \tau \leq 1$$

- Why does this work?
  - When τ is close to 1, the distribution doesn't change much.
  - The lower τ is, the larger the scores being passed to the softmax
  - Softmax pushes high values toward 1 and low values toward 0
  - Large inputs pushes high-probability words higher and low probability word lower, making the distribution "more greedy".
  - As τ approaches 0, the probability of most likely word approaches 1

Temperature scaling



Training scheme: pretraining-finetuning paradigm

- The big idea that underlies all the amazing performance of language models
- First, pretrain a transformer model on enormous amounts of text
- Then apply it to new tasks (after finetuning)

Self-supervised training for pretraining

- We just train them to predict the next word!

1. Take a corpus of text
2. At each time step t
   - ask the model to predict the next word
   - train the model using gradient descent to minimize the error in this prediction

Self-supervised training for pretraining

- We just train them to predict the next word!

1. Take a corpus of text
2. At each time step t
   - ask the model to predict the next word
   - train the model using gradient descent to minimize the error in this prediction

Intuition of language model training: loss

- Same loss function: cross-entropy loss
  - We want the model to assign a high probability to true word w
  - = want loss to be high if the model assigns too low a probability to w

- CE Loss: The negative log probability that the model assigns to the true next word w
  - If the model assigns too low a probability to w
  - We move the model weights in the direction that assigns a higher probability to w

Cross-entropy loss for language modeling

- CE loss: difference between the correct probability distribution and the predicted distribution

$$L_{CE} = -\sum_{w \in V} \mathbf{y}_t[w] \log \hat{\mathbf{y}}_t[w]$$

- The correct distribution $\mathbf{y}_t$ knows the next word, so is 1 for the actual next word and 0 for the others.

- So in this sum, all terms get multiplied by zero except one: the logp the model assigns to the correct next word, so:

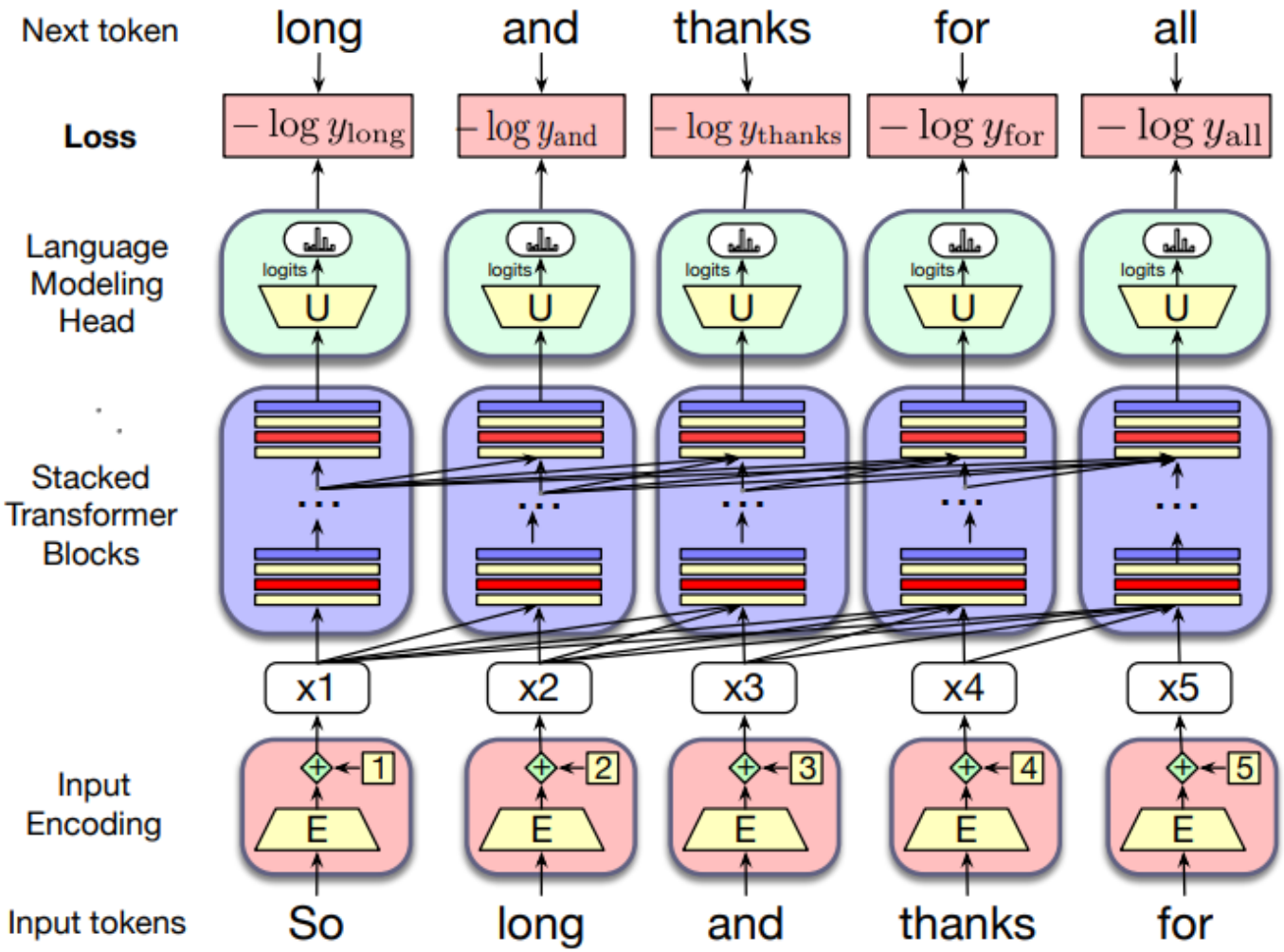
$$L_{CE}(\hat{\mathbf{y}}_t, \mathbf{y}_t) = -\log \hat{\mathbf{y}}_t[w_{t+1}]$$

Teacher forcing

- At each token position t, model sees correct tokens $w_{1:t}$,
  - Computes loss (–log probability) for the next token w_{t+1}

- At next token position $t + 1$, we ignore what model predicted for w_{t+1}
  - Instead, we take the correct word w_{t+1}, add it to context, move on

Training a transformer language model (pretraining)

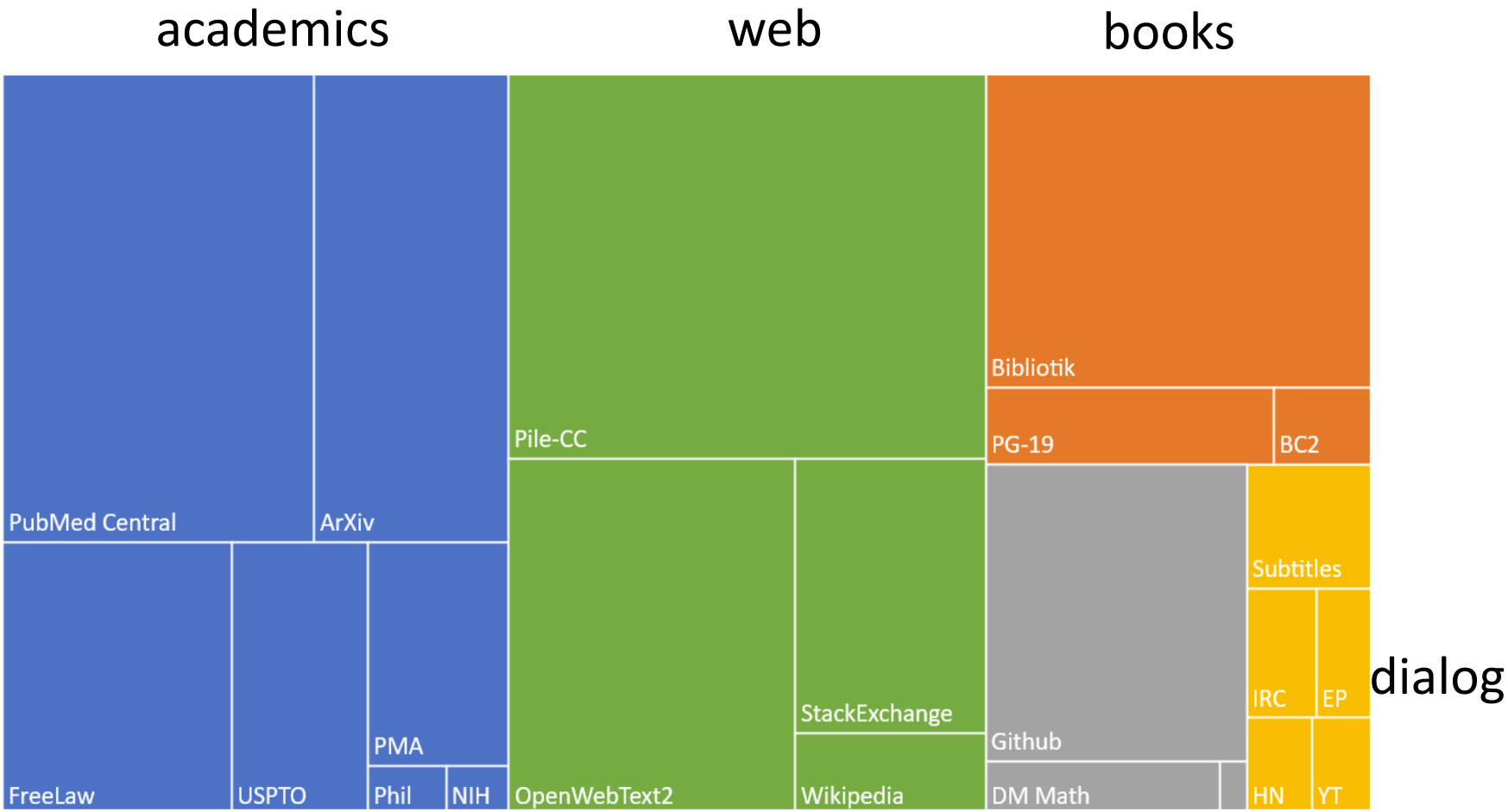


$$= \frac{1}{T}\sum_{t=1}^{T} L_{CE}$$

LLMs are mainly trained on the web

- Snapshots of the entire web produced by the non-profit organization Common Crawl with billions of pages
- Colossal Clean Crawled Corpus (C4; Raffel et al. 2020), 156 billion tokens of English, filtered
- What's in it? Mostly patent text documents, Wikipedia, and news sites

The Pile: a pretraining corpus



Filtering for quality and safety

- Quality is subjective
  - Many LLMs attempt to match Wikipedia, books, particular websites
  - Need to remove boilerplate content
  - Deduplication at many levels (URLs, documents, even lines)

- Safety is also subjective
  - Toxicity detection is important, although that has mixed results
  - Can mistakenly flag data written in dialects like African American English

What does a model learn from pretraining?

- There are dogs everywhere! One dog in the front room, and two dogs
- It wasn't just big it was enormous
- The author of "A Room of One's Own" is Virginia Woolf
- The doctor told me that he
- The square root of 4 is 2

What does a model learn from pretraining?

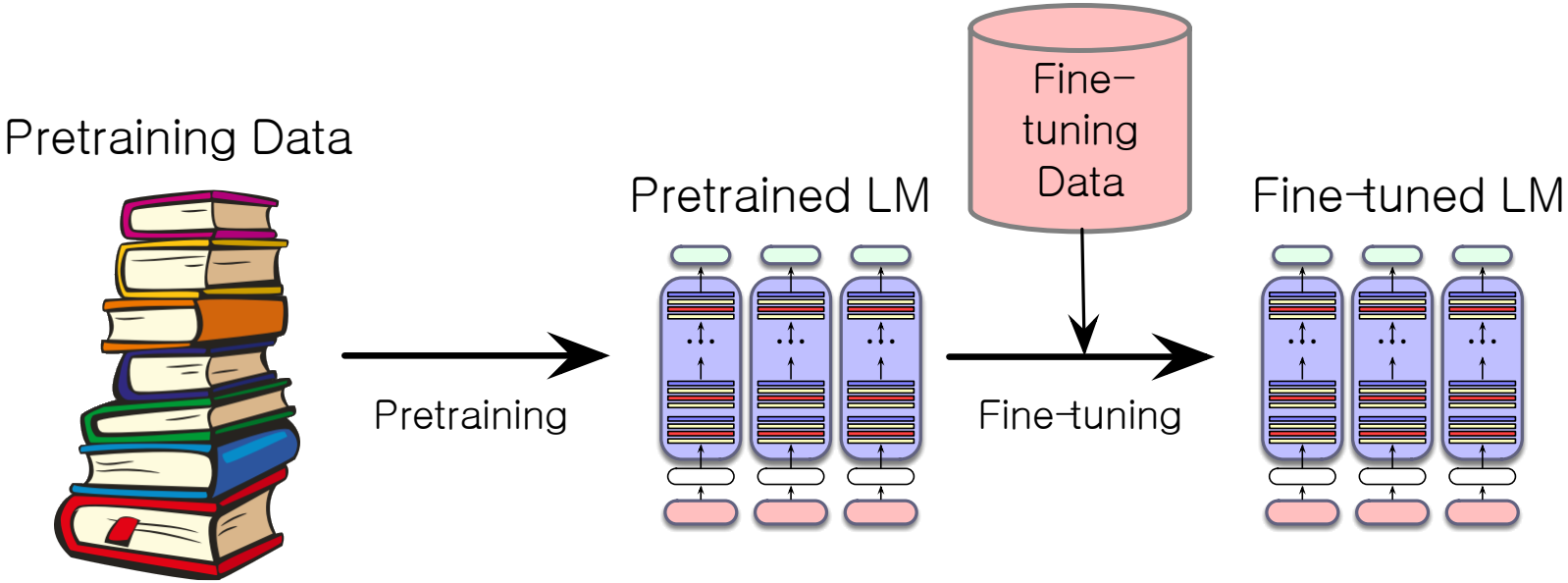
- Text contains enormous amounts of knowledge

- Pretraining on lots of text with all that knowledge is what gives language models their ability to do so much

Finetuning for adaptation to new domains

- What happens if we need our LLM to work well on a domain it didn't see in pretraining?
- Perhaps some specific medical or legal domain?
- Or maybe a multilingual LM needs to see more data on some language that was rare in pretraining?

Pretraining-finetuning paradigm



Finetuning as “continued pretraining” on new data

- Further train all the parameters of model on new data
  - using the same method (word prediction) and loss function (cross-entropy loss) as for pretraining.
  - as if the new data were at the tail end of the pretraining data

- Hence sometimes called **continued pretraining**

Parameter-Efficient Finetuning (PEFT)

- Adapting to a new domain by continued pretraining (finetuning) is a problem with huge LLMs.
  - Enormous numbers of parameters to train
  - Each pass of batch gradient descent has to backpropagate through many many huge layers.
  - Expensive in processing power, in memory, and in time.

- Instead, **parameter-efficient fine tuning** (PEFT)
  - Efficiently select a subset of parameters to update when finetuning.
  - E.g., freeze some of the parameters (don't change them),
  - And only update some few parameters.

LoRA (Low-Rank Adaptation)

- Transformers have many dense matrix multiply layers
  - Like $\boldsymbol{W^Q}, \boldsymbol{W^K}, \boldsymbol{W^V}, \boldsymbol{W^O}$ layers in attention

- Instead of updating these layers during finetuning,
  - Freeze these layers
  - Update a low-rank approximation with fewer parameters.

LoRA (Low-Rank Adaptation)

- Consider a **matrix W** (shape [N × d]) that needs to be updated during finetuning via gradient descent.
  - Normally updates are ΔW (shape [N × d])

- In LoRA, we freeze W and update instead a low-rank decomposition of W:
  - $\boldsymbol{A}$ of shape [N×r],
  - $\boldsymbol{B}$ of shape [r×d], r is very small (like 1 or 2)
  - That is, during finetuning we update $\boldsymbol{A}$ and $\boldsymbol{B}$ instead of $\boldsymbol{W}$.
  - Replace $\boldsymbol{W} + \Delta\boldsymbol{W}$ with $\boldsymbol{W} + \boldsymbol{BA}$.

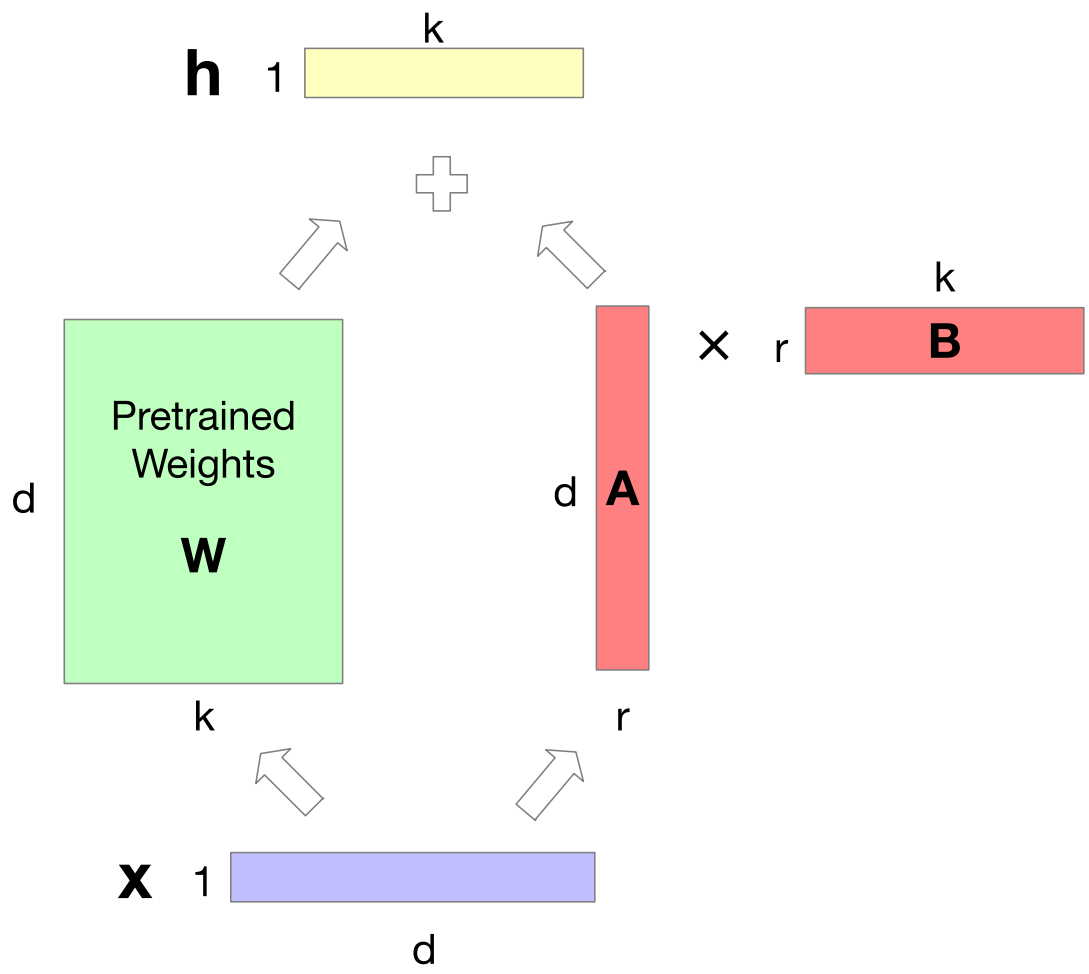
- Forward pass: instead of

$$\boldsymbol{h} = \boldsymbol{xW}$$

- We do

$$\boldsymbol{h} = \boldsymbol{xW} + \boldsymbol{xAB}$$

LoRA (Low-Rank Adaptation)



Unsafe Outputs – Alignment Problem

- LLMs may produce
  - Harmful text – unparliamentary language, bias and discrimination
  - Text that can cause direct harm – allowing easy access to dangerous information

- Therefore, LLMs should be trained to produce outputs that align with human preferences and values

- Modern LLMs do so by using human preference directly in model training

RLHF (Reinforcement Learning from Human Feedback)

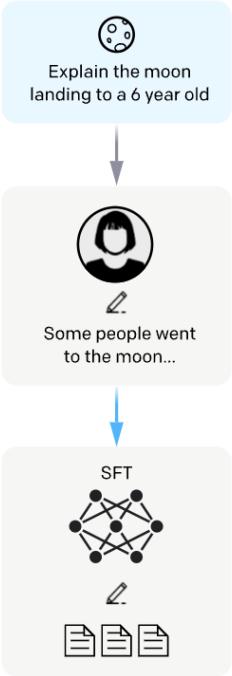
Step 1

Collect demonstration data, and train a supervised policy.

A prompt is sampled from our prompt dataset.

A labeler demonstrates the desired output behavior.

This data is used to fine-tune GPT-3 with supervised learning.



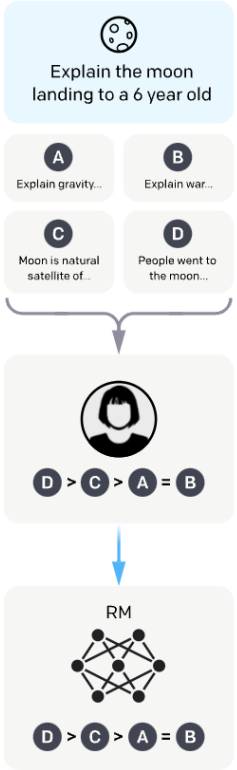
Step 2

Collect comparison data, and train a reward model.

A prompt and several model outputs are sampled.

A labeler ranks the outputs from best to worst.

This data is used to train our reward model.



Step 3

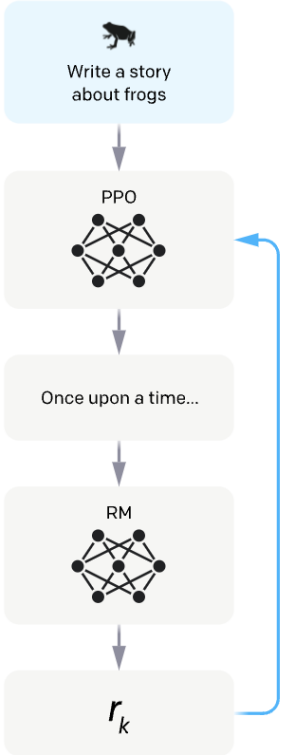
Optimize a policy against the reward model using reinforcement learning.

A new prompt is sampled from the dataset.

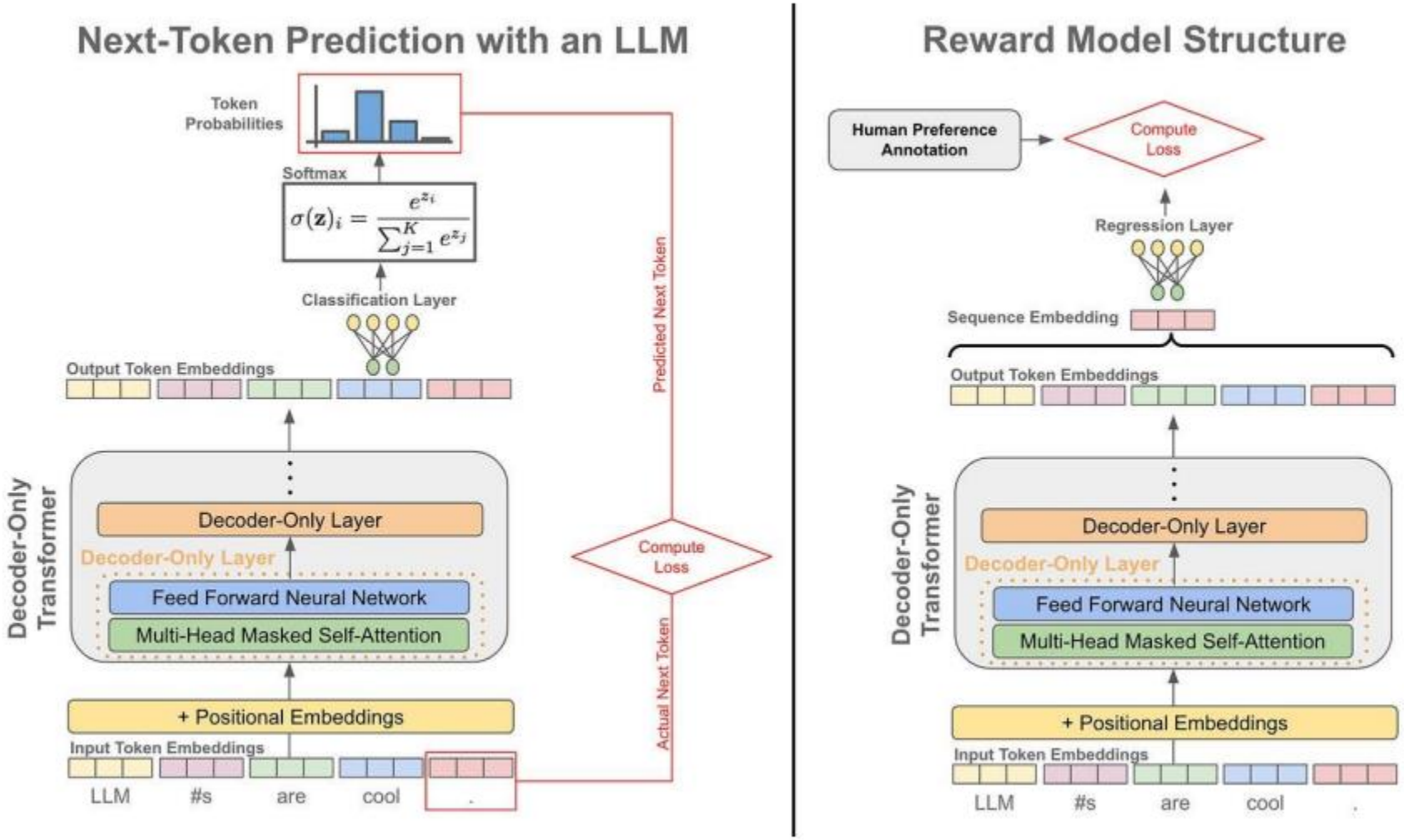
The policy generates an output.

The reward model calculates a reward for the output.

The reward is used to update the policy using PPO.



RLHF (Reinforcement Learning from Human Feedback)



LLM Inference: Prompting

- Remember that LLM is trained to predict next word, given previous words
- Thus, LLM relies on context
- Prompts we give provides context to LLM
- Prompts
  - Tell the model what to do in natural language
  - For example, generate a textual summary of this paragraph:
  - Can be as short or long as required

LLM Inference: Prompting

- Prompt Engineering
  - The task of identifying the correct prompt needed to perform a task
  - General rule of thumb be as specific and descriptive as possible
  - Can be manual or automatic (prefix-tuning, paraphrasing etc.)

In-context learning/ Few-shot prompting

- Provide a few examples along with the instruction

```
Translate English to French:
sea otter => loutre de mer
peppermint => menthe poivrée
plush girafe => girafe peluche
cheese =>
```

task description

examples

prompt

Chain-of-Thought (CoT) Prompting

- Get the model to work through the steps of the problem
  - This is very useful for tasks that require reasoning
  - You can combine it with few-shot prompting to get better results

Standard Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ✗

Chain-of-Thought Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. 5 + 6 = 11. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had 23 - 20 = 3. They bought 6 more apples, so they have 3 + 6 = 9. The answer is 9. ✓

Zero-Shot Chain-of-Thought (CoT) Prompting

- CoT requires human-annotated examples
- Elicits reasoning from LLM by prompting "Let's think step by step"

(a) Few-shot

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?
A: The answer is 11.

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?
A:

(Output) The answer is 8. ✗

(b) Few-shot-CoT

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?
A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. 5 + 6 = 11. The answer is 11.

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?
A:

(Output) The juggler can juggle 16 balls. Half of the balls are golf balls. So there are 16 / 2 = 8 golf balls. Half of the golf balls are blue. So there are 8 / 2 = 4 blue golf balls. The answer is 4. ✓

(c) Zero-shot

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?
A: The answer (arabic numerals) is

(Output) 8 ✗

(d) Zero-shot-CoT (Ours)

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?
A: ***Let's think step by step.***

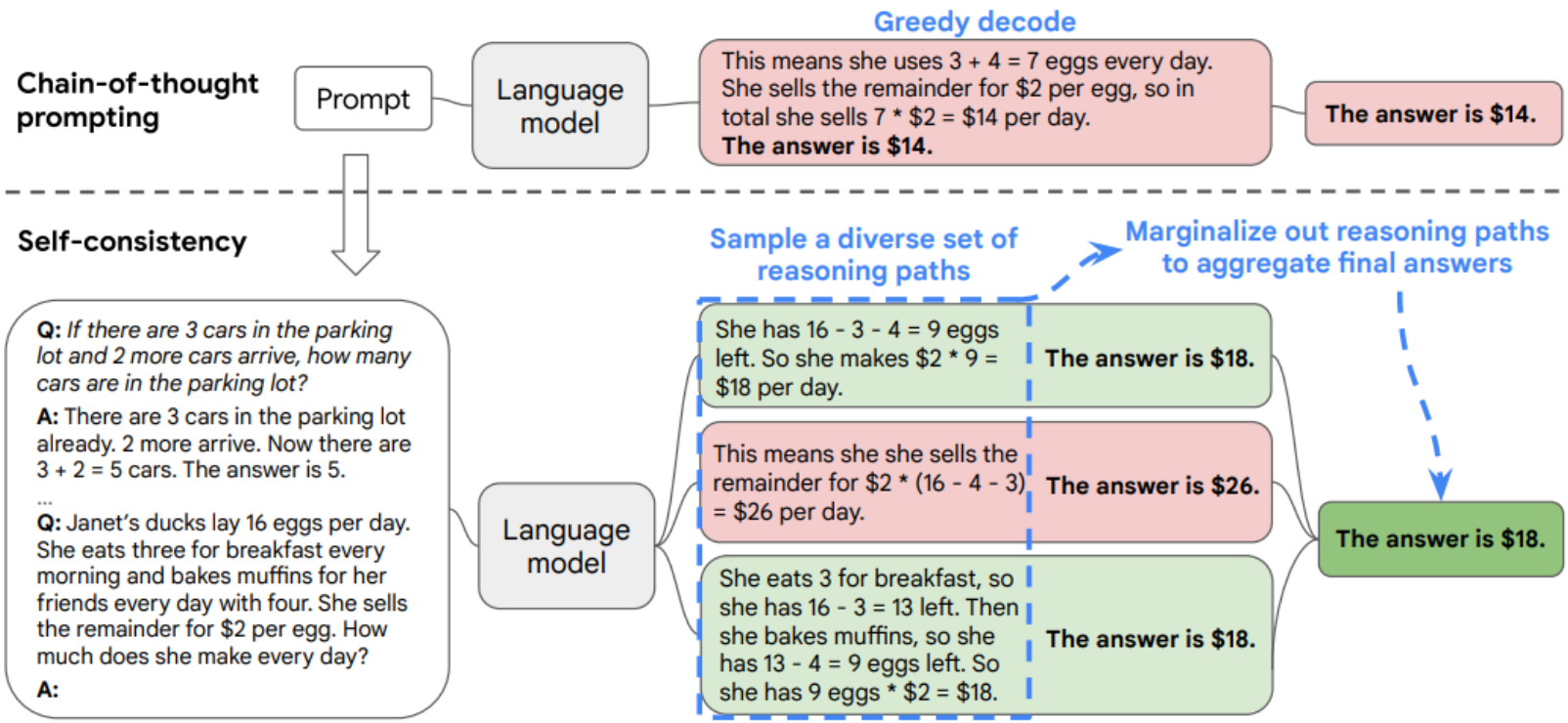
(Output) There are 16 balls in total. Half of the balls are golf balls. That means that there are 8 golf balls. Half of the golf balls are blue. That means that there are 4 blue golf balls. ✓

Zero-Shot Chain-of-Thought (CoT) Prompting

	MultiArith	GSM8K
Zero-Shot	**17.7**	**10.4**
Few-Shot (2 samples)	33.7	15.6
Few-Shot (8 samples)	33.8	15.6
Zero-Shot-CoT	**78.7**	**40.7**
Few-Shot-CoT (2 samples)	84.8	41.3
Few-Shot-CoT (4 samples : First) (*1)	89.2	-
Few-Shot-CoT (4 samples : Second) (*1)	90.5	-
Few-Shot-CoT (8 samples)	93.0	48.7
Zero-Plus-Few-Shot-CoT (8 samples) (*2)	**92.8**	**51.5**

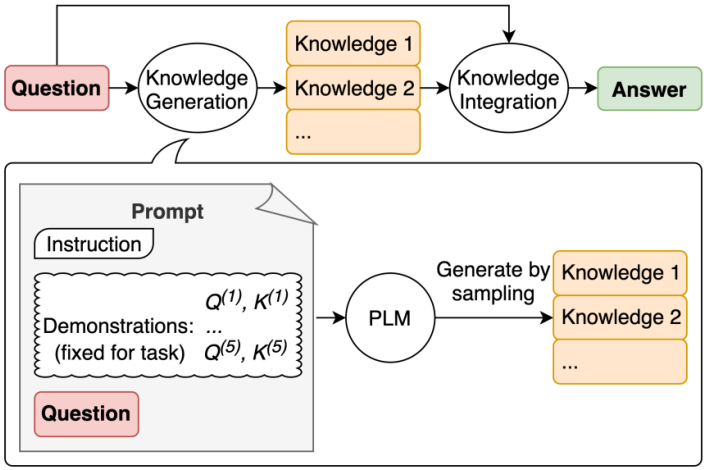
Self-Consistency

- Self-Consistency aims to improve on the naïve greedy decoding used in CoT prompting
- The idea is to sample multiple, diverse reasoning paths through few-shot CoT, and select the most consistent answer



Generate Knowledge Prompting

- This technique involves using additional knowledge provided as part of the context to improve results on complex tasks, such as commonsense reasoning
- The knowledge used in the context is generated by a model and used in the prompt to make a prediction



Prompt Generate some numerical facts about objects. Examples:

Input: **penguins have <mask> wings.**
Knowledge: *Birds have two wings. Penguin is a kind of bird.*

...

Input: **a typical human being has <mask> limbs.**
Knowledge: *Human has two arms and two legs.*

Input: **{question}**
Knowledge:

Program-Aided Language Model (PAL)

- CoT prompting is a good example of how to steer models to perform better at complex reasoning tasks
  - However, sometimes CoT is not enough as it depends only on the generated text from the model
- Program-aided language models (PAL) uses an LLM to read problems and generate programs as the intermediate reasoning steps
  - It offloads the solution step to a runtime such as Python interpreter

Program-Aided Language Model (PAL)

Chain-of-Thought (Wei et al., 2022)

Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 tennis balls. 2 cans of 3 tennis balls each is 6 tennis balls. 5 + 6. The answer is 11.

Q: The bakers at the Beverly Hills Bakery baked 200 loaves of bread on Monday morning. They sold 93 loaves in the morning and 39 loaves in the afternoon. A grocery store returned 6 unsold loaves. How many loaves of bread did they have left?

Model Output

A: The bakers started with 200 loaves. They sold 93 in the morning and 39 in the afternoon. So they sold 93 + 39 = 132 loaves. The grocery store returned 6 loaves. So they had 200 - 132 - 6 = 62 loaves left.
The answer is 62.

Program-aided Reasoning (this work)

Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 tennis balls.
`tennis_balls = 5`
2 cans of 3 tennis balls each is
`bought_balls = 2 * 3`
tennis balls. The answer is
`answer = tennis_balls + bought_balls`

Q: The bakers at the Beverly Hills Bakery baked 200 loaves of bread on Monday morning. They sold 93 loaves in the morning and 39 loaves in the afternoon. A grocery store returned 6 unsold loaves. How many loaves of bread did they have left?

Model Output

A: The bakers started with 200 loaves
`loaves_baked = 200`
They sold 93 in the morning and 39 in the afternoon
`loaves_sold_morning = 93`
`loaves_sold_afternoon = 39`
The grocery store returned 6 loaves.
`loaves_returned = 6`
The answer is

```
answer = loaves_baked - loaves_sold_morning
  - loaves_sold_afternoon + loaves_returned
```

```
>>> print(answer)
74
```

ReAct

- ReAct is a framework where LLMs are used to generate both reasoning traces and task-specific actions in an interleaved manner
  - **Generating reasoning traces** allow the model to induce, track, and update action plans, and even handle exceptions
  - **The action step** allows to interface with and gather information from external sources, such as knowledge bases or environments

- ReAct allows LLMs to interact with external tools to retrieve additional information that leads to more reliable and factual responses

ReAct

(1) Hotspot QA

Question: Aside from the Apple Remote, what other device can control the program Apple Remote was originally designed to interact with?

(1a) Standard

Answer: iPod ✗

(1b) CoT (Reason Only)

Thought: Let's think step by step. Apple Remote was originally designed to interact with Apple TV. Apple TV can be controlled by iPhone, iPad, and iPod Touch. So the answer is iPhone, iPad, and iPod Touch.

Answer: iPhone, iPad, iPod Touch ✗

(1c) Act-Only

Act 1: Search[Apple Remote]
Obs 1: The Apple Remote is a remote control …

Act 2: Search[Front Row]
Obs 2: Could not find [Front Row]. Similar: …

Act 3: Search[Front Row (software)]
Obs 3: Front Row is a discontinued media center software …

Act 4: Finish[yes] ✗

(1d) ReAct (Reason + Act)

Thought 1: I need to search Apple Remote and find the program it was originally designed to interact with.
Act 1: Search[Apple Remote]
Obs 1: The Apple Remote is a remote control introduced in October 2005 by Apple … originally designed to control the Front Row media center program …

Thought 2: Apple Remote was originally designed to control the Front Row media center program. I need to search Front Row next and find what other device can control it.
Act 2: Search[Front Row]
Obs 2: Could not find [Front Row]. Similar: ['Front Row Seat to Earth', 'Front Row Motorsports',' Front Row (software)', …]

Thought 3: Front Row is not found. I need to search Front Row (software) .
Act 3: Search[Front Row (software)]
Obs 3: Front Row is a discontinued media center software …

Thought 4: Front Row (software) is controlled by an Apple Remote or the keyboard function keys. So the answer is keyboard function keys.
Act 4: Finish[keyboard function keys] ✓

Evaluation of LLM: Perplexity

- We use perplexity to measure how well the LM predicts unseen text

- The perplexity of a model θ on an unseen test set is the **inverse probability that θ assigns to the test set, normalized by the test set length.**

- For a test set of *n* tokens $w_{1:n}$ the perplexity is :

$$\begin{aligned}\text{Perplexity}_\theta(w_{1:n}) &= P_\theta(w_{1:n})^{-\frac{1}{n}} \\ &= \sqrt[n]{\frac{1}{P_\theta(w_{1:n})}}\end{aligned}$$

Why perplexity instead of raw probability of the test set?

- Probability depends on size of test set
  - Probability gets smaller the longer the text
  - Better: a metric that is **per-word**, normalized by length

- **Perplexity** is the inverse probability of the test set, normalized by the number of words
  - (The inverse comes from the original definition of perplexity from cross-entropy rate in information theory)
  - Probability range is [0,1], perplexity range is [1,∞]

Why perplexity instead of raw probability of the test set?

- The higher the probability of the word sequence, the lower the perplexity.
- Thus the lower the perplexity of a model on the data, the better the model.
- **Minimizing perplexity is the same as maximizing probability**

- Also: perplexity is sensitive to length/tokenization so best used when comparing LMs that use the same tokenizer.

Why perplexity instead of raw probability of the test set?

- The higher the probability of the word sequence, the lower the perplexity.
- Thus the lower the perplexity of a model on the data, the better the model.
- **Minimizing perplexity is the same as maximizing probability**

- Also: perplexity is sensitive to length/tokenization so best used when comparing LMs that use the same tokenizer.

Other factors for evaluation

- Size
  - Big models take lots of GPUs and time to train, memory to store
- Energy usage
  - Can measure kWh or kilograms of CO2 emitted
- Fairness
  - Benchmarks measure gendered and racial stereotypes, or decreased performance for language from or about some groups.

Scaling Laws

- With scaling laws, we can make decisions on architecture, data, hyperparameters by training smaller models

- Open AI Study : Scaling Lawsfor Neural Language Models(Kaplan et al. 2020)

- Key Findings:
  - Performance depends strongly on scale, and weakly on the model shape
  - Larger models are more sample-efficient
  - Smooth power laws between empirical performance and N - parameters, D - dataset size, C - compute

Scaling Laws

- The effect of some hyperparameters on big LMs can be predicted before training – optimizer (Adam v/s SGD), model depth, LSTM v/s Transformer

- Idea:
  - Train a few smaller models
  - Establish a scaling law (e.g. ADAM vs SGD scaling law)
  - Select optimal hyper param based on the scaling law prediction

Scaling Laws

- LLM performance depends on
  - Model size: the number of non-embedding parameters
  - Dataset size: the amount of training data
  - Compute: Amount of compute (in FLOPS or etc)

- Can improve a model by adding parameters (more layers, wider contexts), more data, or training for more iterations

- The performance of a large language model (the loss) scales as a power-law with each of these three

Scaling Laws

- Loss *L* as a function of # parameters *N*, dataset size *D*, compute budget *C (*if other two are held constant)

$$
\begin{aligned}
L(N) &= \left(\frac{N_c}{N}\right)^{\alpha_N} \\
L(D) &= \left(\frac{D_c}{D}\right)^{\alpha_D} \\
L(C) &= \left(\frac{C_c}{C}\right)^{\alpha_C}
\end{aligned}
$$

- Scaling laws can be used early in training to predict what the loss would be if we were to add more data or increase model size

Number of non-embedding parameters N

$$
\begin{aligned}
N &\approx 2\, d\, n_{\text{layer}} (2\, d_{\text{attn}} + d_{\text{ff}}) \\
&\approx 12\, n_{\text{layer}}\, d^2 \\
&\quad (\text{assuming } d_{\text{attn}} = d_{\text{ff}}/4 = d)
\end{aligned}
$$

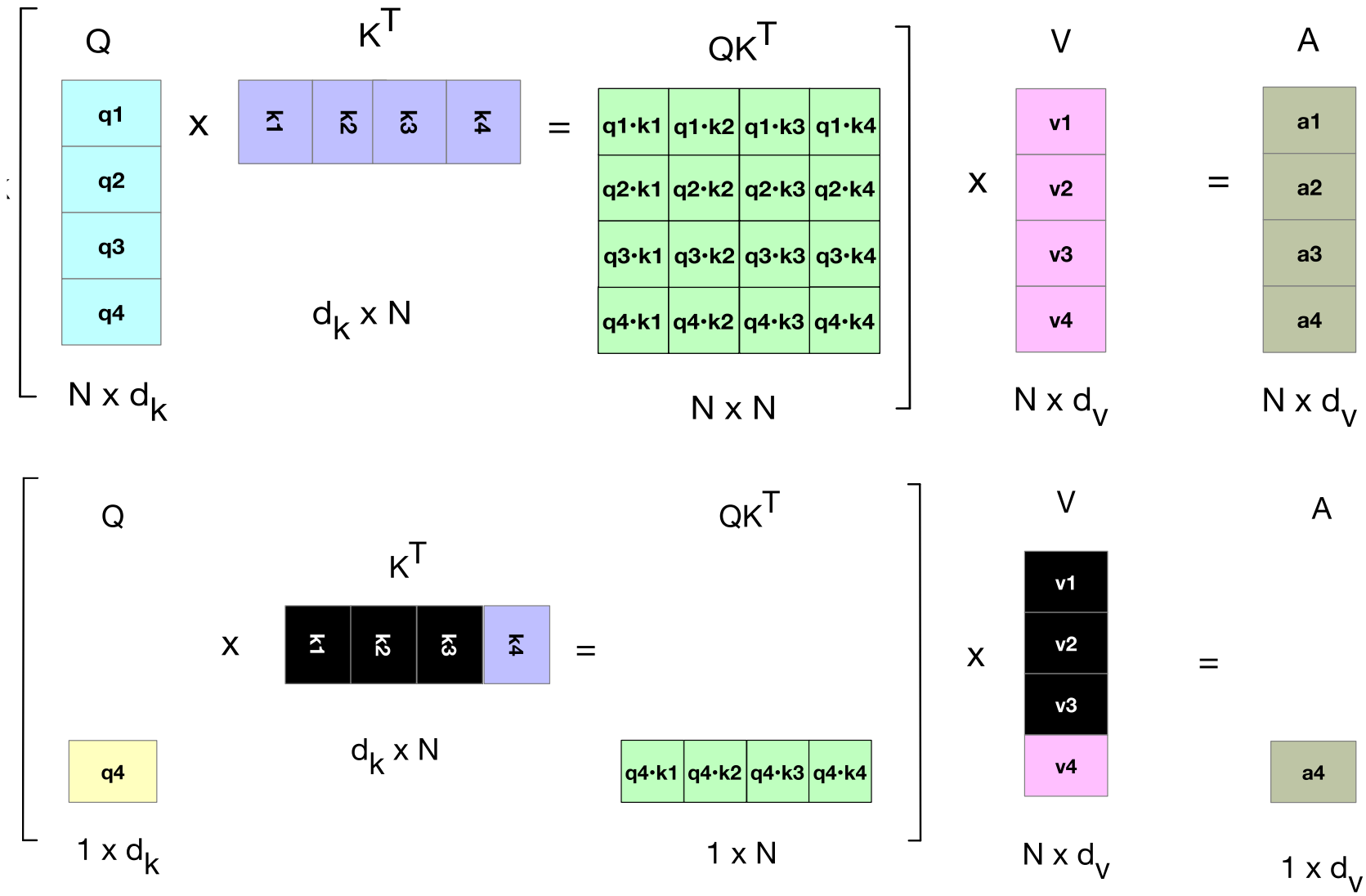
KV Cache

- In training, we can compute attention very efficiently in parallel:

$$\mathbf{A} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\intercal}{\sqrt{d_k}}\right)\mathbf{V}$$

- But not at inference! We generate the next tokens **one at a time!**
- For a new token x, need to multiply by W^Q, W^K, and W^V to get query, key, values
- But don't want to **recompute** the key and value vectors for all the prior tokens $x_{<i}$
- Instead, store key and value vectors in memory in the KV cache, and then we can just grab them from the cache

KV Cache


Q
K^T
QK^T
V
A
q1
q2
q3
q4
x
k1
k2
k3
k4
=
q1·k1 q1·k2 q1·k3 q1·k4
q2·k1 q2·k2 q2·k3 q2·k4
q3·k1 q3·k2 q3·k3 q3·k4
q4·k1 q4·k2 q4·k3 q4·k4
x
v1
v2
v3
v4
=
a1
a2
a3
a4
N x d_k
d_k x N
N x N
N x d_v
N x d_v
Q
K^T
QK^T
V
A
x
k1
k2
k3
k4
=
v1
v2
v3
v4
=
q4
q4·k1 q4·k2 q4·k3 q4·k4
a4
1 x d_k
d_k x N
1 x N
N x d_v
1 x d_v

But there are problems with scraping from the web

- Copyright: much of the text in these datasets is copyrighted
  - Not clear if fair use doctrine in US allows for this use
  - This remains an open legal question
- Data consent
  - Website owners can indicate they don't want their site crawled
- Privacy
  - Websites can contain private IP addresses and phone number

Hallucination

Chatbots May 'Hallucinate' More Often Than Many Realize

What Can You Do When A.I. Lies About You?

People have little protection or recourse when the technology creates and spreads falsehoods about them.

Air Canada loses court case after its chatbot hallucinated fake policies to a customer

The airline argued that the chatbot itself was liable. The court disagreed.

Copyright

Authors Sue OpenAI Claiming Mass Copyright Infringement of Hundreds of Thousands of Novels

The Times Sues OpenAI and Microsoft Over A.I. Use of Copyrighted Work

Millions of articles from The New York Times were used to train chatbots that now compete with it, the lawsuit said.

Privacy

Toxicity and Abuse

The New AI-Powered Bing Is Threatening Users.

Cleaning Up ChatGPT Takes Heavy Toll on Human Workers

Contractors in Kenya say they were traumatized by effort to screen out descriptions of violence and sexual abuse during run-up to OpenAI's hit chatbot

Misinformation

Chatbots are generating false and misleading information about U.S. elections